Gestion des Processus, Ordonnancement et Signaux POSIX sous Linux

Thème principal : Systèmes d'exploitation et Fonctionnement (Unix/Linux)

Synthèse exécutive : Ce cours pose les fondements indispensables de la gestion des tâches sous les systèmes d'exploitation de type Unix/Linux. Il détaille la transition critique entre un programme passif stocké sur disque et un processus dynamique résidant en mémoire vive (RAM). Pour garantir l'exécution simultanée et fluide de multiples applications, le noyau s'appuie sur le multitâche préemptif et un cycle de vie à trois états fondamentaux : Élu, Prêt et Bloqué. La séance aborde également l'arborescence généalogique des processus (PID/PPID) issue du processus racine systemd/init (PID 1), ainsi que la communication asynchrone par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'un ordinateur exécute des instructions, le système d'exploitation doit faire basculer le code d'un état statique (sur le disque) à un état dynamique (en mémoire vive). Le noyau gère cette exécution en orchestrant les processus à travers un cycle de vie rigoureux. Lorsqu'un processus est à l'état 'Élu', il utilise activement le processeur. Si son quantum de temps expire, l'ordonnanceur le replace dans la file d'attente à l'état 'Prêt'. S'il effectue une opération d'entrée/sortie (comme lire un fichier sur le disque ou attendre qu'un utilisateur tape du texte), il passe à l'état 'Bloqué' afin de ne pas gaspiller inutilement des cycles CPU. Une fois l'opération terminée, le noyau le bascule d'abord à l'état 'Prêt' et non directement à l'état 'Élu'. Tous les processus descendent d'une structure arborescente stricte dont la racine absolue est le processus systemd ou init (PID 1).

- Programme vs Processus : Différence entre le fichier binaire passif sur disque et l'instance active en RAM.
- Multitâche préemptif : Allocation et retrait de quanta de temps CPU par l'ordonnanceur.
- Bloc de Contrôle du Processus (PCB) : Structure noyau regroupant PID, PPID, états et registres.
- Filiation PID / PPID : Hiérarchie des processus issue du processus racine systemd/init (PID 1).
- Les trois états fondamentaux : Élu (Running), Prêt (Ready) et Bloqué (Waiting).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme stocké sur le disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (statique, passif, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail/RAM, du temps de cuisson/CPU et manipulant des ustensiles/descripteurs de fichiers).*

Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est émis vers un processus cible, le noyau interrompt momentanément son fil d'exécution normal pour le forcer à réagir. Bien que le développeur puisse écrire du code pour intercepter (catch) ou ignorer la majorité des signaux, la norme POSIX impose des garde-fous stricts : les signaux SIGKILL (9) et SIGSTOP (19) ne peuvent jamais être interceptés ou masqués. Ils sont traités directement par le noyau pour donner à l'administrateur le pouvoir absolu de reprendre le contrôle d'une machine face à des processus récalcitrants.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau.
- Gestionnaire de signal (Signal Handler) : Fonction logicielle pour intercepter (catch) un signal.
- SIGINT (Signal 2) : Interruption interactive générée par Ctrl + C.
- SIGTERM (Signal 15) : Demande de terminaison propre par défaut.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré par le noyau, inintercepteable.
- Job Control (SIGSTOP 19 / SIGCONT 18) : Suspension et reprise de processus.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de quitter les lieux (vous disposez du temps nécessaire pour ranger vos affaires, sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (9) est une intervention d'urgence des forces de sécurité qui défonce la porte et vous expulse sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Parce que sans le tiret '-', le shell interprète '9' comme le premier argument positionnel (le signal) et '1234' comme un PID, mais la commande kill s'attend à ce que le signal soit préfixé par un tiret (ex: -9 ou -SIGKILL) pour le différencier clairement du PID cible.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus zombie est un processus terminé dont le parent n'a pas encore lu le code de retour via l'appel système wait(). Il ne consomme plus de CPU ni de mémoire RAM (sauf une entrée dans la table des processus), et il est impossible de le tuer avec un 'kill -9' car il est... déjà mort ! La solution consiste à éliminer son processus parent pour que systemd adopte le zombie et nettoie automatiquement sa table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de la première ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche immédiatement l'identifiant de tâche ainsi que le PID attribué (ex: [1] 4521) et redonne la main. Étape 2 : La commande 'ps -ef | grep sleep' interroge tous les processus du système et filtre les résultats pour afficher la ligne correspondant à 'sleep' afin de relever son PID exact (4521). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM au processus. L'utilitaire 'sleep' applique le comportement par défaut : il libère ses ressources et s'arrête proprement.

Résultat : Le processus 'sleep 600' est définitivement arrêté de manière propre. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus actifs du système.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux lors du démarrage de la machine ?

R : systemd (ou init), avec le PID 1.

Q : Quel signal POSIX est généré et transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' dans son terminal ?

R : Le signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme informatique ?

R : Parce qu'il est traité directement par le noyau Linux pour garantir à l'administrateur un moyen infaillible de stopper un processus bloqué ou malveillant.